Hand-Written Notes

On Kubernetes Resource Quota (A Simple strong Student Guide)

What is Resource Quota?

Resource Quota is a method in Kubernetes to **control resource usage** at the project level (**Namespace** level).

There are two types of Resource Quota:

1. **Computing Level Quota** – Controls CPU and RAM (Memory) usage.
2. **Object Level Quota** – Limits the number of **Pods, Deployments, Services, PVCs**, etc.

Why is Resource Quota Applied at the Namespace Level?

In Kubernetes, every resource is created inside a **Namespace**. A **Namespace** defines the boundary for project resources. So, **Resource Quota is always set at the Namespace level** to manage resource limits effectively.

Why is Resource Quota Important?

Imagine a **Kubernetes Cluster** where multiple teams work together. If one team uses **too many resources**, other teams **may not get enough**. So **Resource Quota** prevents this by ensuring fair distribution.

Problem: One team uses all the resources.
Solution: Quota limits resource usage per team.

Example:

- Team A (Amazon) gets limited resources.
- Team B (Facebook) gets its own fair share.

Namespaces and Resource Quota

In Kubernetes, a **Namespace** divides a cluster into smaller sections, providing **logical isolation** at the project level. When a **Resource Quota** is applied to a Namespace, it ensures that all resources within that Namespace adhere to the defined limits. However, it does **not** impact other projects, as each operates in its own separate Namespace.
For example, if a **cluster has a total capacity of 32GB RAM and 16 CPU Cores**, the administrator can allocate resources as follows:

- **Team A** gets **20GB RAM and 10 CPU Cores**.
- **Team B** gets **10GB RAM and 4 CPU Cores**.
- **2GB RAM and 2 CPU Cores** are reserved for future use.

Example:

Team A → Namespace = amazon
Team B → Namespace = facebook
Each team can create resources **only within their assigned Resource Quota**, ensuring fair distribution of cluster resources.

Types of Resource Quota

As discussed earlier, there are two main types of Resource Quota:

1. **Computing Quota** – Controls CPU and Memory (RAM).
2. **Object Quota** – Limits the number of Kubernetes objects like **Pods, Deployments, Services, PVCs**, etc.

Type-1 (Object Resource Quota) – Kubernetes object Resources

How Does Object Quota Work?

A **Object Quota** sets a **limit on the maximum resources** that can be used in a **Namespace**. It controls computing resource like **CPU, RAM, storage, and the number of objects (Pods, Services, Deployments, etc.)**.

Example: Setting a Object Quota for the "AMAZON" Namespace

```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: team-a-quota
spec:
  hard:
    pods: "10"            # Maximum 10 Pods
    services: "5"         # Maximum 5 Services
    deployments: "10"     # Maximum 10 deployments
    secrets: "15"         # Maximum 15 Secrets
    requests.cpu: "2"     # Max 2 CPU requests
    requests.memory: "4Gi"  # Max 4GB RAM requests
    limits.cpu: "4"         # Max 4 CPU usage
    limits.memory: "8Gi"    # Max 8GB RAM usage
    requests.storage: "100Gi"        # Max 100GB Storage
    persistentvolumeclaims: "5"      # Max 5 PVCs
```

What Does This Mean for the Namespace? When this Resource Quota is applied:

- A maximum of 10 Pods can be created.

- CPU and Memory usage cannot exceed the defined limits.
- Storage usage cannot exceed 100GB.
- Only 5 PVCs (Persistent Volume Claims) can be created.
- Only 15 Secrets can be stored.
- Only 10 Deployments are allowed.

What if limit try to crossed (How Does Resource Quota Work?)

- Admin sets a Resource Quota inside a Namespace.
- Teams create resources (Pods, Services) inside the Namespace.
- Kubernetes checks if the request follows the Quota limits.
- If the request exceeds the limit, Kubernetes blocks it.

Example: If team-a tries to create an **11th Pod**, it will get an **error:**

Error: exceeded quota: team-a-quota, requested: pods=1, used: pods=10, limited: pods=10

Meaning: Cannot create more than 10 Pods**.**

Type-2 (Compute Resource Quota) – CPU & Memory Limits

Compute Quota is a type of Resource Quota that specifically controls CPU and Memory (RAM) usage in a Namespace. It ensures that no team or application overuses computing resources, keeping the cluster balanced and efficient.

Resource Quota controls CPU and Memory in two ways:

Requests – The minimum CPU and RAM a Pod needs.
Limits – The maximum CPU and RAM a Pod can use.

How Does Compute Quota Work? Step by Step

1. **Admin sets a Compute Quota in a Namespace.**
2. **Teams create Pods and other resources.**
3. **Kubernetes checks if CPU and Memory requests are within limits.**
4. **If a request exceeds the limit, Kubernetes blocks it.**

Example: Setting a Compute Quota

```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: compute-quota
spec:
  hard:
    requests.cpu: "2"  # Minimum 2 CPU required
    requests.memory: "4Gi"  # Minimum 4GB RAM required
    limits.cpu: "4"  # Maximum 4 CPU allowed
    limits.memory: "8Gi"  # Maximum 8GB RAM allowed
```

What This Means:

- Minimum CPU a Pod can request: **Up to 2 CPUs in total.**
- Minimum RAM a Pod can request: **Up to 4GB in total.**
- Maximum CPU a Pod can use: **Up to 4 CPUs in total.**
- Maximum RAM a Pod can use: **Up to 8GB in total.**

Important:

- **requests.cpu** & **requests.memory** → Define minimum guaranteed resources for Pods.
- **limits.cpu** & **limits.memory** → Define maximum allowed usage for Pods.

Meaning:

- **CPU Requests** = Minimum CPU allocation.
- **CPU Limits** = Maximum CPU allocation.

What is compute measurement unit:

1 CPU = 1000m (millicores)
1 GB RAM = 1024Mi

What Happens If Limits Are Exceeded?

Example: If a Pod requests more than 2 CPUs or tries to use more than 4 CPUs, Kubernetes will block the request and show an error.

Error: exceeded quota: compute-quota, requested: cpu=3, used: cpu=2, limited: cpu=2

Meaning: The request exceeds the allowed CPU limit in this Namespace.

Priority-Based Quota – High-Priority Pods

Sometimes, **important Pods** need extra resources. For this, **Priority-Based Quota** is used.

Why is Priority-Based Quota Used?

Imagine you are running a **shared Kubernetes cluster** where multiple teams and applications use the same resources. Some applications are more important than others. For example:

- A **payment service** must always work, even if resources are low.
- A **report generation service** can wait if needed.

To **ensure that critical applications always get the resources they need**, **Priority-Based Quota** is used.

How It Works (In Simple Terms)

- **Admins assign different priority levels** (High, Medium, Low).
- **Each priority level gets a fixed amount of resources (CPU, RAM, Pods).**

- **High-priority applications always get their reserved resources first.**
- **If resources are low, low-priority applications may be stopped or delayed.**

Real-Life Example

Think of it like **boarding a flight**:

1. **High-Priority (Business Class)** – Always gets a seat, even if the flight is full.
2. **Medium-Priority (Premium Economy)** – Gets a seat if available.
3. **Low-Priority (Economy Class)** – May be denied a seat if the flight is overbooked.

Similarly, **Kubernetes ensures high-priority applications always run**, while low-priority applications may be limited if resources are tight.

Example of Priority-Based Quota (Commented for Easy Understanding)

```yaml
apiVersion: v1
kind: List
items:
- apiVersion: v1
  kind: ResourceQuota
  metadata:
    name: pods-high          # Resources reserved for high-priority applications
  spec:
    hard:
      cpu: "1000"            # High-priority apps can use up to 1000 CPUs
      memory: "200Gi"        # Can use up to 200GB RAM
      pods: "10"             # Only 10 high-priority pods allowed
    scopeSelector:
      matchExpressions:
      - operator: In
        scopeName: PriorityClass
        values: ["high"]     # Applied only to high-priority pods

- apiVersion: v1
  kind: ResourceQuota
  metadata:
    name: pods-medium        # Resources reserved for medium-priority applications
  spec:
    hard:
      cpu: "10"              # Medium-priority apps can use up to 10 CPUs
      memory: "20Gi"         # Can use up to 20GB RAM
      pods: "10"             # Only 10 medium-priority pods allowed
    scopeSelector:
      matchExpressions:
      - operator: In
        scopeName: PriorityClass
        values: ["medium"]   # Applied only to medium-priority pods

- apiVersion: v1
```

```yaml
kind: ResourceQuota
metadata:
  name: pods-low            # Resources reserved for low-priority applications
spec:
  hard:
    cpu: "5"                # Low-priority apps can use up to 5 CPUs
    memory: "10Gi"          # Can use up to 10GB RAM
    pods: "10"              # Only 10 low-priority pods allowed
  scopeSelector:
    matchExpressions:
    - operator: In
      scopeName: PriorityClass
      values: ["low"]       # Applied only to low-priority pods
```

Meaning:

- High-Priority Pods get their own limits.
- Up to 10 High-Priority Pods are allowed.
- High-priority apps always get resources first.
- Low-priority apps may be paused if resources are low.
- Ensures fair and efficient resource usage in Kubernetes.

How to Use Priority-Based Resource Quota in a Deployment or Pod?

To apply **Priority-Based Resource Quota** to a **Deployment or Pod**, you need to use **PriorityClass**.

- **Use PriorityClass in a Deployment**
  Now, apply **PriorityClass** in a **Deployment** to ensure it gets the right resource quota.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: high-priority-app
  namespace: example-namespace
spec:
  replicas: 2
  selector:
    matchLabels:
      app: important-app
  template:
    metadata:
      labels:
        app: important-app
    spec:
      priorityClassName: high-priority        # Assigning high-priority
      containers:
      - name: app-container
        image: nginx
        resources:
          requests:
```

```
        cpu: "500m"                    # Requesting 0.5 CPU
        memory: "512Mi"                # Requesting 512MB RAM
      limits:
        cpu: "1"                       # Max usage 1 CPU
        memory: "1Gi"                  # Max usage 1GB RAM
```

- **Use PriorityClass in a Pod (Without Deployment)**
  If you want to use **PriorityClass** in a **Pod directly**, use this:

```
apiVersion: v1
kind: Pod
metadata:
  name: medium-priority-pod
  namespace: example-namespace
spec:
  priorityClassName: medium-priority  # Assigning medium-priority
  containers:
  - name: app-container
    image: nginx
    resources:
      requests:
        cpu: "200m"
        memory: "256Mi"
      limits:
        cpu: "500m"
        memory: "512Mi"
```

Kubernetes Updated on Resource Quota (Things you have to know)

Extended Resource Quota

Since Kubernetes 1.10, **Resource Quota** has been extended to support **special hardware resources** like **GPU, FPGA, and other custom hardware**.

For example, if you want to limit the number of GPUs used in a **Namespace**, you can define a quota like this:

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: gpu-quota
spec:
  hard:
        requests.cpu: "4" requests.memory: "16Gi"
        requests.nvidia.com/gpu: "4"  # Maximum 4 GPUs can be used in namespace
```

Meaning:

- The Namespace where this quota is applied **cannot use more than 4 GPUs**.
- This feature is useful for workloads that require **machine learning, AI processing, or high-performance computing** while ensuring fair resource distribution.

Storage Resource Quota

Storage Resource Quota is part of the **ResourceQuota object**, but it specifically deals with **storage-related limits**, unlike standard **object quotas** that manage Kubernetes objects (like Pods, Services, and PVC counts).

Let's break it down clearly:

Where does Storage Resource Quota fit?

- **Object Quota:** Controls **how many** Kubernetes objects (Pods, PVCs, Services, etc.) can exist.
- **Compute Quota:** Controls **how much** CPU, Memory, and special resources (like GPU) can be used.
- **Storage Quota:** Controls **how much storage (Persistent Volume Claims - PVCs)** can be used.

```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: my-resource-quota
  namespace: my-namespace        # Replace with your actual namespace
spec:
  hard:
    pods: "10"                   # Max 10 Pods allowed
    services: "5"                # Max 5 Services allowed
    persistentvolumeclaims: "5"  # Max 5 PVCs allowed
    requests.cpu: "2"            # Max 2 CPU requests
    requests.memory: "4Gi"       # Max 4GB RAM requests
    limits.cpu: "4"              # Max 4 CPU usage
    limits.memory: "8Gi"         # Max 8GB RAM usage
    requests.storage: "100Gi"    # Max 100GB total storage usage
    configmaps: "10"             # Max 10 ConfigMaps allowed
    secrets: "15"                # Max 15 Secrets allowed
```

Kubernetes Resource Quota Assignment – Professional Quiz

Instructions:

This quiz is designed to test your understanding of **Kubernetes Resource Quotas** (Computing Quota & Object Quota) in real-world scenarios. Read each question carefully and think about the best possible answer before proceeding.

---

1. Understanding Namespace-Level Resource Quotas

- Your company manages multiple teams working on different projects within a shared Kubernetes cluster. To prevent one team from consuming excessive resources, you decide to implement Resource Quotas.
- **Why is Resource Quota always applied at the Namespace level instead of the cluster level? How does this help in managing resources effectively?**

2. **Handling Resource Limits for Pods**
    - A development team is working in a Namespace where the following quota is applied:

      pods: "10" Now when They attempt to deploy an 11th Pod.
    - **What will happen in this case? How does Kubernetes enforce this quota limit?**

---

3. **Compute Resource Quota Enforcement**

- A Kubernetes administrator has set up the following Compute Resource Quota for a Namespace:

  requests.cpu: "4"

  limits.cpu: "8"

  requests.memory: "8Gi"

  limits.memory: "16Gi"

  pods: "5"

- Currently, the team has already created **5 Pods**, but there is still CPU and Memory available.
- **Can they create an additional Pod? Why or why not?**

---

4. **Monitoring and Troubleshooting Resource Quotas**
    - As a Kubernetes administrator, you need to monitor the current resource usage in a specific Namespace and identify whether it is approaching its quota limits.
    - **Which kubectl command would you use to check the existing Resource Quota and its current usage?**

---

5. **Requests vs. Limits in Compute Quotas**
    - Your team is configuring a **Compute Resource Quota** for their Namespace, but they are confused about the difference between requests and limits.
    - **In a Kubernetes Compute Quota, what is the difference between "requests" and "limits" for CPU and Memory? Why is this distinction important?**

---

6. **Object Count Quota Enforcement**
    - A Namespace has the following Object Quota applied:

      pods: "10"

      services: "5"

      secrets: "15"
    - Currently, the team has deployed **9 Pods, 5 Services, and 10 Secrets**.

- **How many more Pods, Services, and Secrets can they create before hitting the quota limit?**

---

7. Handling Storage Resource Quotas

- A team is deploying applications that require Persistent Volumes. Their Namespace has the following Storage Resource Quota:
  requests.storage: "500Gi"
  persistentvolumeclaims: "10"
- Currently, they have **6 PersistentVolumeClaims (PVCs)**, each requesting **50Gi** of storage.
- **Can they create 5 more PVCs, each requesting 50Gi of storage? Why or why not?**

---

8. Impact of Resource Quotas on High-Priority Pods

- Your team is deploying High-Priority Pods in a cluster that already has a strict Resource Quota applied. However, the High-Priority Pods are getting rejected due to quota restrictions.
- **How can you configure Resource Quotas to ensure High-Priority Pods get the necessary resources even in a constrained environment?**

---

9. Debugging Quota-Related Issues

- A developer reports that their Pod creation request is failing due to a quota limit error. You need to investigate the issue.
- **Which Kubernetes command would you use to check recent quota-related errors and understand why the request is failing?**

---

10. Scaling and Resource Quotas

- Your team is planning to scale an application by adding more Pods. However, they discover that the Namespace has hit its Resource Quota limits.
- **What are the possible ways to allow more Pods to be created while maintaining fair resource distribution among teams?**

Essential Kubernetes Commands for Managing Resource Quotas

- **Create a Compute Resource Quota (CPU & Memory) from Command Line**

```
kubectl create resourcequota compute-quota --namespace=<namespace> \
  --hard=requests.cpu=2,limits.cpu=4,requests.memory=4Gi,limits.memory=8Gi
```

Creates a quota that limits CPU and Memory usage in the specified Namespace.

- **Create an Object Count Quota (Pods, Services, PVCs) from Command Line**

```
kubectl create resourcequota object-quota --namespace=<namespace> \
  --hard=pods=10,services=5,persistentvolumeclaims=5,secrets=15
```

Creates a quota that limits the number of Kubernetes objects like Pods, Services, PVCs, and Secrets.

- **List All Resource Quotas in a Namespace**

```
kubectl get resourcequota --namespace=<namespace>
```

Displays all existing Resource Quotas in the specified Namespace.

- **View Detailed Information About a Specific Resource Quota**

kubectl describe resourcequota <quota-name> --namespace=<namespace>

Shows quota limits, current resource usage, and remaining capacity.

- **Check Node-Level Resource Quota**

```
kubectl describe node <node-name>
```

Shows CPU, memory, and other allocated resources at the **node level**.

- **Check the Current Resource Usage in a Namespace**

```
kubectl top pod --namespace=<namespace>
```

Displays real-time CPU and memory usage of Pods.

- **Apply a New Resource Quota from a YAML File**

```
kubectl apply -f <resource-quota.yaml> --namespace=<namespace>
```

Creates or updates a Resource Quota from a YAML definition.

- **Edit an Existing Resource Quota**

```
kubectl edit resourcequota <quota-name> --namespace=<namespace>
```

Opens the quota configuration for editing in a text editor.

- **Delete a Resource Quota**

```
kubectl delete resourcequota <quota-name> --namespace=<namespace>
```

Removes the Resource Quota from the Namespace.

- **Check Recent Quota-Related Errors (Events)**

```
kubectl get events --namespace=<namespace>
```

Lists all recent events, including Resource Quota violations.

- **Verify Resource Requests and Limits for a Pod**

```
kubectl get pod <pod-name> -o yaml --namespace=<namespace>
```

Displays detailed YAML output to check if the Pod is within quota limits.

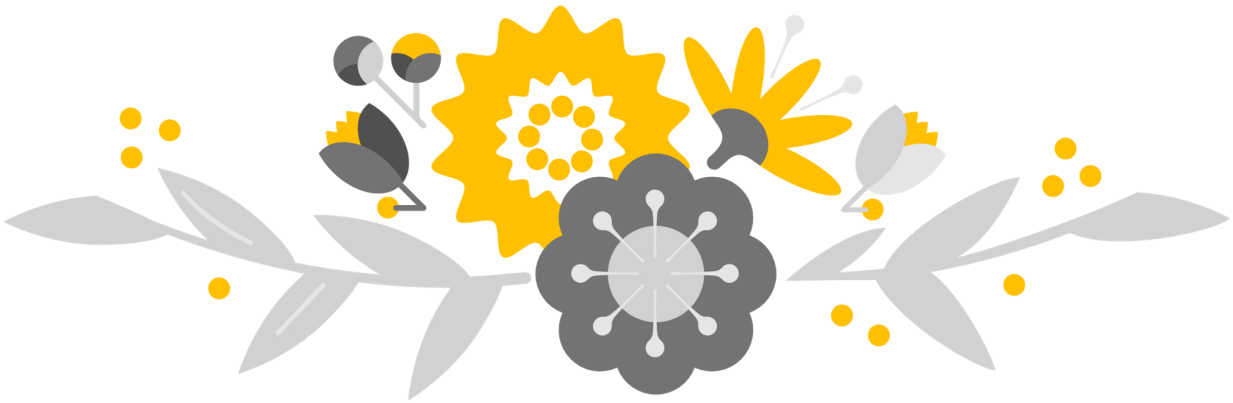
- **Simulate a Quota Violation Before Applying a Deployment**

  kubectl create -f <deployment.yaml> --dry-run=server --namespace=<namespace>

Checks if a new deployment would exceed the quota without actually applying it.

- **Increase Resource Quota Limits (By Applying an Updated YAML)**

  kubectl apply -f updated-quota.yaml --namespace=<namespace>

Allows increasing limits when more resources are required.

Follow this channel for more: **https://www.linkedin.com/in/rakeshkumarjangid/**